

Scientific Report for DSPRO2 Course (FS25)

Beyond2D

Generate 3D models from 2D images

Fabian Dubach <fabian.dubach@stud.hslu.ch>

Timon Schmid <timon.schmid@stud.hslu.ch>

Yannick Frischknecht <yannick.frischknecht@stud.hslu.ch>

Rotkreuz, June 2025



Figure 1: Visualization showcasing the reconstruction of a complete and detailed 3D interior from multiple 2D source images.

Contents

1. Introduction	3
1.1. Project idea	3
1.2. Problem definition	3
1.3. Motivation	3
1.4. Objectives	3
1.5. Relevance	3
2. Literature review	4
2.1. General information	4
2.2. Single-view and Multi-view 3D reconstruction	4
2.3. 3D reconstruction methods	4
2.4. Comparison of 3D reconstruction methods	6
2.5. Enhancing SfM/MVS with AI: Our Optimization Strategy	7
3. Data	8
3.1. Datasets	8
3.2. Data Analysis	9
3.3. Data Cleaning	12
3.4. Preprocessing	15
3.5. Data Splitting	15
4. Methods	16
4.1. SfM/MVS Pipeline	16
4.2. Monocular Depth Estimation	20
5. Results	23
5.1. Performance	23
5.2. Visualizations	24
6. Model Integration	25
6.1. How COLMAP Works	25
6.2. Integration Approach	25
6.3. Results Basic DepthAnythingV2	26
6.4. Results Trained DepthAnythingV2	27
6.5. Cleaning Up the Results	28
6.6. Creating the Final Meshes	29
6.7. Putting Together Complete Apartments	29
7. Machine Learning Operations	30
7.1. Version Control	30
7.2. Cloud Computing	30
7.3. Experiment Tracking	30
8. Conclusions	31
8.1. Interpretation of Results	31
8.2. Interpretation of Model Integration	31
8.3. Future Research Directions	32
8.4. Summary	32
9. Glossary	33
10. References	34

1. Introduction

1.1. Project idea

Our project covers the 3D reconstruction of apartments using image data. Specifically, we aim to integrate a self-trained monocular depth estimation model into an existing Multi-View Stereo (MVS) pipeline. This approach will allow us to enhance the accuracy and robustness of 3D models generated from limited image inputs.

1.2. Problem definition

When browsing apartment listings online, users are typically limited to 2D images that lack depth information. These flat images can be misleading, especially when wide-angle lenses distort the space. Without a clear sense of scale or layout, it's difficult for potential renters or buyers to understand the true dimensions of the space. Our project addresses this issue by reconstructing the apartment in 3D, providing a more accurate and immersive view.

1.3. Motivation

The real estate market increasingly relies on digital platforms, yet the viewing experience remains largely two-dimensional. Virtual tours and 3D visualizations are becoming more in demand as people seek more convenient, informative and remote ways to evaluate properties. Our motivation is to improve the digital viewing experience and make property assessments more accurate and user-friendly.

1.4. Objectives

For our project we have the following objectives:

- Train a monocular depth estimation model specifically tailored for indoor scenes.
- Integrate this model into an existing MVS reconstruction pipeline to improve depth information from sparse or uncalibrated image sets.
- Evaluate the impact of our depth estimator on reconstruction quality compared to baselines.
- Produce 3D models from the dense point cloud from the SfM/MVS pipeline.

1.5. Relevance

This project is relevant to fields such as real estate, virtual reality and computer vision. In particular, our contribution to enhance an existing MVS pipeline with a learned monocular depth estimation, addresses a real-world need: generating spatially accurate 3D environments from collected imagery from apartments. This makes the technology accessible and useful for real estate listings, interior design planning and virtual property viewings.

2. Literature review

2.1. General information

3D reconstruction is the process of using 2D images or sensor data to create various 3D models. Generally, 3D reconstruction helps computers to understand the shape and structure of the physical world. This technology can be applied in various fields such as cultural heritage preservation, medical imaging, robotics, CAD's, virtual and augmented reality. In cultural heritage, 3D reconstruction is used to digitally preserve artifacts and sites, which is important for studying and protecting them against potential extinction. In medical imaging it helps in visualizing anatomical structures, enhancing diagnostic accuracy and even surgical planning. Robotics applies 3D reconstruction, especially in fields like environmental mapping and navigation, whereas CAD is used for creating accurate and detailed digital models of real-world objects or environments. In virtual and augmented reality, 3D reconstruction is applied to create immersive and interactive experiences. As technology improves, 3D reconstruction is becoming faster, more accurate and more widely used in everyday applications. [1]

2.2. Single-view and Multi-view 3D reconstruction

3D reconstruction methods can be classified into single-view and multi-view approaches, depending on whether they only use one or multiple images as input.

Single-view 3D reconstruction tries to create a 3D model from just one 2D image. This process is very challenging because many 3D shapes can look the same in 2D. Traditional methods rely on cues like shading or texture, but need strong assumptions and don't work well in real life. Deep learning methods have improved single-view 3D reconstruction results by learning 3D shapes from single images, but still face limitations in accuracy and require a lot of training data. [2]

Multi-view reconstruction uses several images shot from different angles, allowing more accurate and complete 3D models. Classical methods match features across images to estimate camera positions and build 3D shapes. The problem with these methods is that they often struggle with occlusions and widely spaced views. Recent deep learning techniques combine multiple views to improve results, though some methods can be slow or lose details. [2]

2.3. 3D reconstruction methods

There are a lot of different 3D reconstruction methods, each with its advantages and disadvantages. These methods range from traditional geometric techniques like Structure from Motion (SfM) to modern deep learning approaches such as Generative Adversarial Networks (GANs) and Neural Radiance Fields (NeRF). Exploring and understanding all these different methods provides a foundation for generating 3D models from 2D images and helps us decide which method fits our use case the best.

2.3.1. Structure from Motion (SfM) with Multi-View Stereo (MVS)

SfM is a photogrammetric technique that reconstructs 3D models utilizing overlapping photographs taken from various angles. With this approach, no expensive equipment (e.g. LiDAR camera) is needed. Often, it's enough to use consumer-grade digital cameras or phones.

The pipeline for SfM begins by identifying distinctive features across images. The algorithm then finds keypoints in images that stay the same even if the object changes in size, rotation, lighting or is partly hidden. This helps match features between different images more reliably. Once features are detected, the algorithm matches these points across multiple images to find correspondences.

Using the matched features, SfM estimates the relative position and orientation (pose) of each camera when the images were captured. This step is crucial as it establishes the spatial relationship between the different views.

With the camera poses and matched features, SfM triangulates the 3D positions of the points seen in multiple images, resulting in a sparse point cloud that outlines the basic structure of the scene.

Techniques such as bundle adjustment are applied to refine these estimates and reduce errors. [3]

To enhance this reconstruction, Multi-View Stereo (MVS) is used to generate a dense point cloud. MVS builds on the output of SfM by exploiting pixel-level correspondences across multiple images to estimate depth maps for each view. These depth maps are then fused into a dense 3D representation. Various MVS approaches, such as plane-sweeping, patch matching or learning-based depth estimation, are designed to improve accuracy, completeness and robustness, especially in challenging regions like textureless surfaces or occlusions. The result is a highly detailed 3D model including surface geometry and texture information suitable for visualization, measurement or further processing. [4]

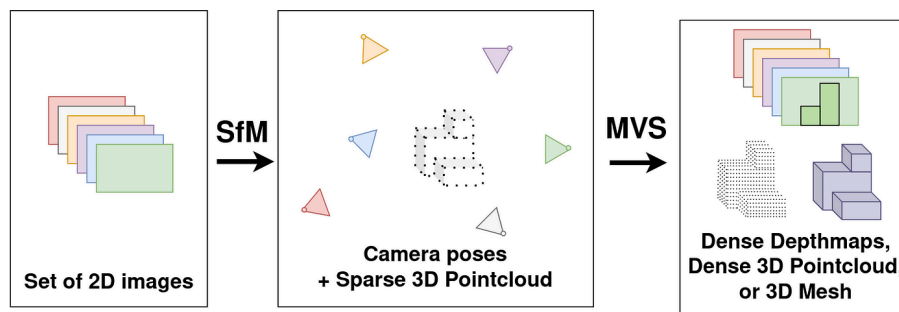


Figure 2: Structure from Motion and Multi-View Stereo

2.3.2. Generative Adversarial Networks (GANs)

GANs are deep learning models consisting of two parts: a generator and a discriminator. The generator creates fake data and the discriminator tries to tell the difference between real and fake data. As they train together, the generator gets better at making realistic data, while the discriminator improves at spotting the fakes. This back-and-forth helps the model create high-quality, realistic results.

For 3D reconstruction, GANs can learn to model detailed 3D shapes from training data without needing labels. They can generate 3D objects in formats like voxels, point clouds or meshes. The generator takes a compressed input (called a latent vector) and turns it into a 3D shape. The discriminator checks whether the shape looks realistic and has a proper structure.

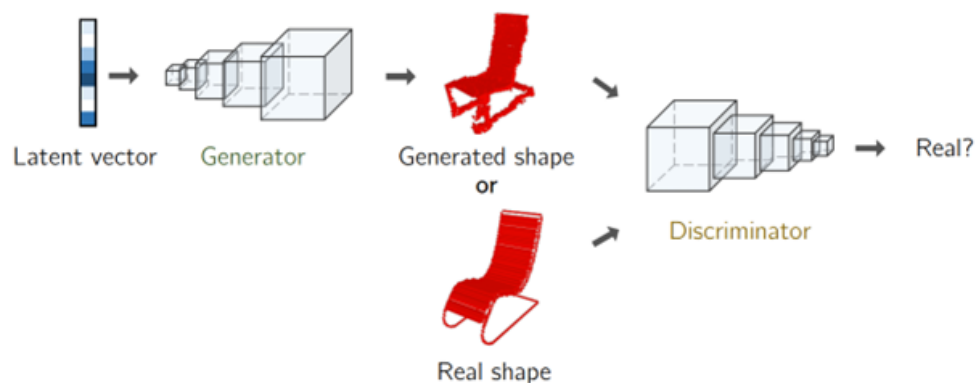


Figure 3: NeRF Learning Schematic

GANs can also explore the space of possible shapes, making it possible to smoothly change one shape into another or edit object features, like switching styles or categories. They're useful for tasks like filling in missing shape parts, creating more training data, or recovering 3D shapes from images. Because they can learn from unlabeled data and generate consistent 3D results, GANs are a valuable tool for 3D reconstruction based on data. [5]

2.3.3. Neural Radiance Fields (NeRF)

NeRF is a method for creating realistic 3D scenes, not necessarily 3D models, using a set of 2D images. It works by learning a function that maps 3D positions and viewing directions to color and density. This function is built using a neural network, which takes five inputs (5-dimensional): the 3D coordinates (x, y, z) and the 2D viewing direction (θ, ϕ) . The network then predicts the color and density at that point in the scene.

To create a new view, NeRF sends rays from the camera into the scene and samples points along each ray. The network predicts color and density for each point and these values are combined using a technique called volume rendering to get the final color of each pixel. Since this process is differentiable, the model can be trained by comparing the predicted images with the real ones and adjusting itself to improve accuracy.

To help the network capture fine details, NeRF uses positional encoding. It also uses hierarchical sampling, which focuses more on important areas of the scene. With these techniques, NeRF can generate high-quality new views of scenes, showing detailed shapes, reflections and transparent surfaces. [6]

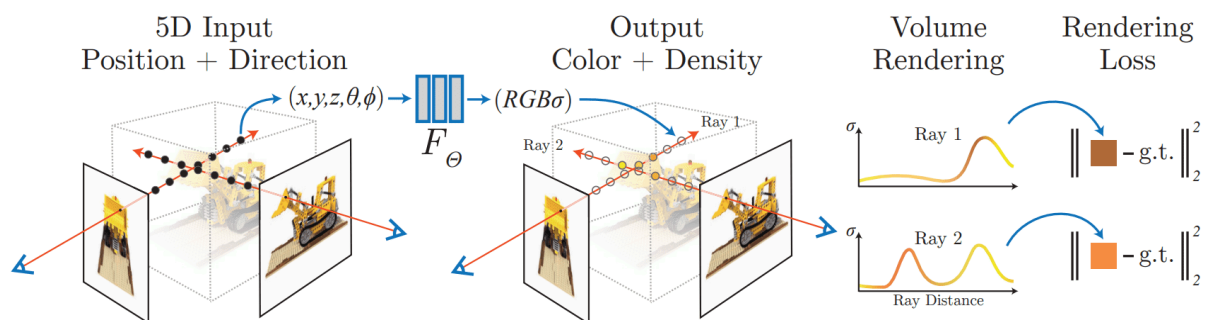


Figure 4: NeRF Learning Schematic

2.4. Comparison of 3D reconstruction methods

For our project, we needed to evaluate which of the three 3D reconstruction methods would be most suitable for our use case. To support this decision, we identified and compared the advantages and disadvantages of each approach.

The SfM/MVS pipeline is flexible and scalable, working with unordered image collections taken under various conditions. It produces explicit geometry like point clouds or meshes, which are usable in standard 3D software. It also runs on standard consumer hardware without needing powerful GPUs. However, it struggles with reflective or transparent surfaces and may produce incomplete reconstructions in areas with little texture or poor image coverage. The multi-stage pipeline can also be complex, with potential failure points during processing.

GANs can directly convert one or more input images into full 3D models without separate preprocessing steps. GANs are capable of generating fine texture details and can work with relatively small training datasets. They are well-suited for creating new shapes or completing partial objects. On the downside, GANs are difficult to train and often unstable, requiring careful tuning. They also tend to be limited in resolution, with many models only supporting low-resolution

outputs. Additionally, training GANs is computationally expensive and time-consuming, especially for high-quality results.

NeRF can render highly realistic novel views with detailed lighting and reflections, even from a small number of 2D input images. This makes NeRF ideal for applications focused on photorealistic view synthesis. Nevertheless, it is very computationally intensive, often requiring long training times and powerful GPUs. Rendering is also slow, as each pixel requires many calculations. Furthermore, for each scene you need to train a new NeRF model and cannot easily generalize to new ones without retraining.

We chose SfM combined with MVS for our project because it offers a good balance between flexibility, scalability and practical usability. While the other two methods offer high visual quality, they require more computational resources and are harder to implement for general-purpose reconstruction tasks. SfM/MVS provided a reliable and efficient solution for our needs.

2.5. Enhancing SfM/MVS with AI: Our Optimization Strategy

To enhance the quality and completeness of our 3D reconstructions, we aim to improve the MVS component of our pipeline by integrating a fine-tuned monocular depth estimation model.

Traditional MVS approaches often struggle with indoor environments. To address this, we plan to train and fine-tune a state-of-the-art monocular depth estimation network on a dataset of indoor scenes. By leveraging deep learning's ability to infer depth from single images, this model will serve as a strong prior, providing initial dense depth maps that can guide or supplement the MVS process. These depth predictions can be used to initialize or refine the depth hypotheses in MVS, improving both convergence and output quality, especially in textureless or low-light areas.

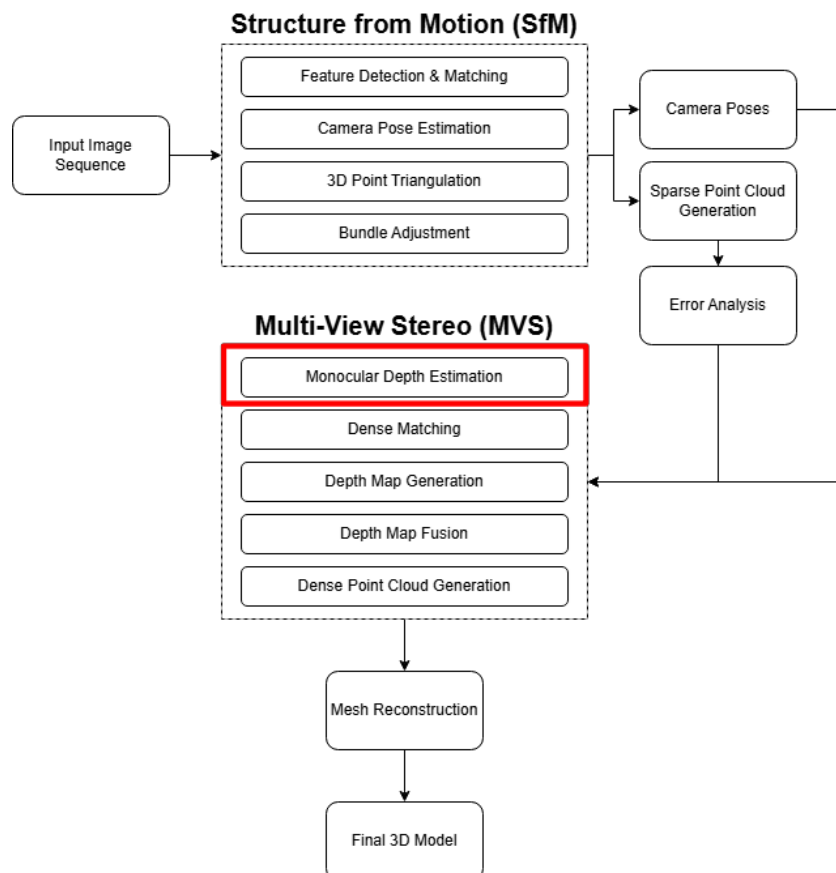


Figure 5: SfM/MVS pipeline with highlighted Monocular Depth Estimation

3. Data

3.1. Datasets

3.1.1. DeepFurniture

For fine-tuning our depth estimation model, we used the DeepFurniture dataset [7] by COOHOM/酷家乐. DeepFurniture is a large-scale dataset designed for furniture understanding, containing a rich collection of photo-realistic rendered indoor scenes, which is crucial for training models for indoor environments.

The DeepFurniture dataset consists of approximately 24'000 photo-realistic indoor images. These scenes were all rendered using COOHOM's render engine and contain 170'000 annotated furniture instances derived from 20'000 unique 3D furniture models. Furthermore, the furniture is categorized into 11 types and 11 distinct styles.

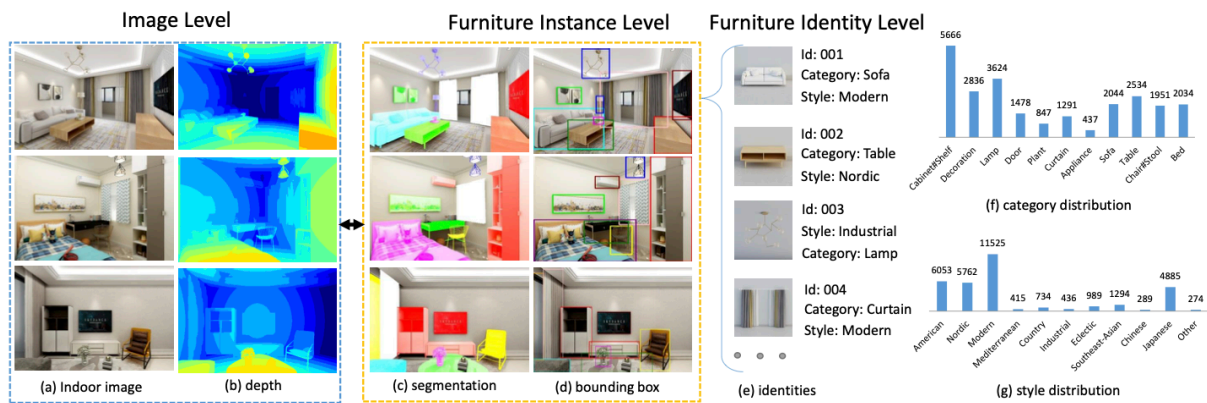


Figure 6: Overview of contents in DeepFurniture dataset

The key strength of DeepFurniture is its multiple annotation levels:

- **Image Level:** Provides the rendered indoor scenes along with their categories and, crucially for our work, depth maps.
- **Instance Level:** Includes bounding boxes and segmentation masks for individual furniture items within the scenes.
- **Identity Level:** Offers high-quality rendered previews of the 3D furniture models.

3.1.2. Living Room Dataset

We use the living room dataset [8] to test and evaluate our SfM/MVS pipeline. The dataset provides a synthetic indoor scene rendered in high fidelity, with images generated from tightly spaced camera viewpoints. This is a key requirement for successful SfM, which relies on strong feature overlap and consistent viewpoint transitions. We chose this dataset primarily for its convenience: it allows us to bypass the time-consuming process of capturing real-world image sequences while still providing a realistic testbed with fine-grained control.

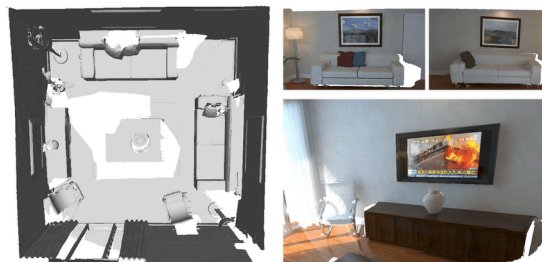


Figure 7: Overview of contents in the Living Room dataset

3.1.3. Unused Datasets

3.1.3.1. NYU Depth V2

Another dataset we initially wanted to use is NYU Depth V2 [9], but we later decided against it due to it already being used in training DepthAnythingV2. This dataset consists of video sequences captured by Microsoft's Kinect camera in various indoor scenes.

It features 1,449 densely labeled pairs of aligned RGB and depth images. These images span 464 distinct indoor scenes taken from three different cities. An important characteristic of the labeled depth data is that it has been preprocessed to fill in missing depth values using the colorization scheme by Levin et al., providing complete depth maps. While the dataset also contains a much larger set of unlabeled frames (407,024) and raw Kinect data, our focus would have been on these aligned RGB-D pairs. The labeled data is provided as a Matlab .mat file and includes per-pixel class and instance labels in addition to the depth information.

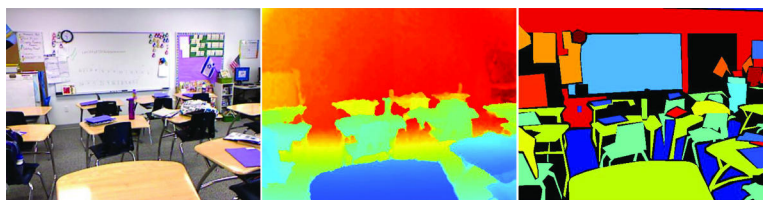


Figure 8: Overview of contents in NYU Depth V2 dataset

For each labeled pair, there is the original RGB image (left), the depth map (middle), and a labeled image (right).

3.2. Data Analysis

To better understand the characteristics of the DeepFurniture dataset, we computed the following statistical metrics for each depth map:

- **min_depth**: Minimum depth value in the image
- **max_depth**: Maximum depth value in the image
- **mean_depth**: Average depth value
- **std_depth**: Standard deviation of depth values
- **median_depth**: Median depth value
- **depth_range**: Difference between the maximum and minimum depth values

These metrics were chosen to give a clear and useful summary of each depth map. Since the dataset contains around 24,000 images, it's not possible to load all of them into memory at once for analysis. To solve this, we calculated summary statistics for each image individually. This allowed us to create a smaller, more memory-efficient dataset that we could use for analysis and visualization.

3.2.1. Histogram

Histograms were generated for all six metrics to explore their distributions across the dataset:

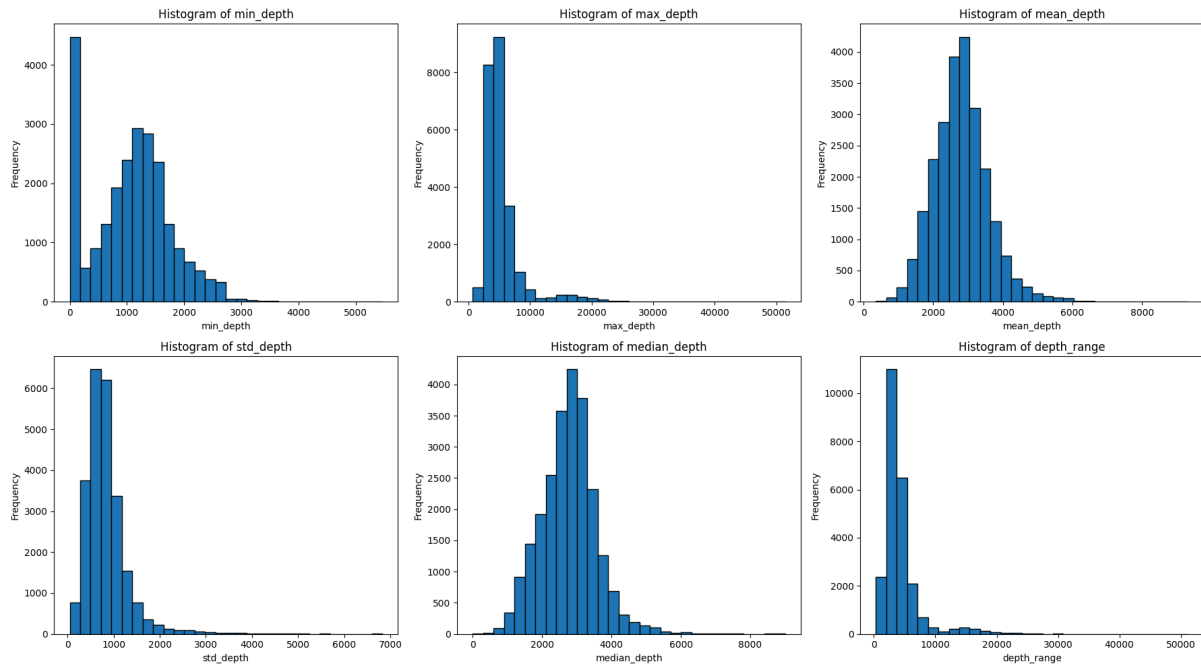


Figure 9: Histogram showing depth distribution across the dataset

mean_depth: The distribution is moderately skewed, with most depth maps centered around typical average depth values, suggesting consistency in scene scale.

std_depth: A right skew shows differences in depth complexity. Some scenes have little variation (like flat or uniform surfaces), while others have a lot, suggesting more detailed spatial structure.

depth_range: Displays a long-tailed distribution, with most depth maps having moderate range and a few exhibiting extremely large depth spreads, likely due to scenes combining close and distant elements.

min_depth: Peaks near zero, implying that many images contain very close foreground elements or sensor artifacts.

max_depth: While many depth maps reach similar maximum values, the long right tail reveals the presence of far background elements in certain scenes.

median_depth: Follows a similar shape to the mean depth distribution but with slightly less spread, reflecting typical scene depth while minimizing the impact of extreme values.

3.2.2. Boxplot

Box plots were created for all metrics to provide a concise summary of their variability and to detect outliers:

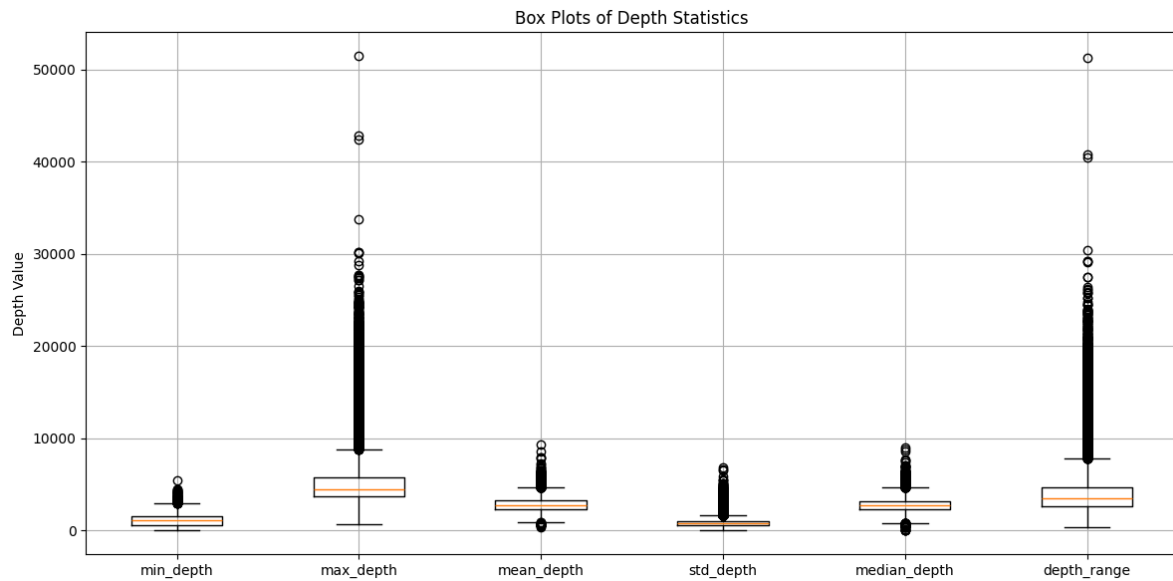


Figure 10: Boxplots showing depth variability and outliers across the dataset

min_depth: Shows low values for most images, with some outliers indicating closer-than-typical foreground objects.

max_depth: Exhibits the largest number of extreme outliers, consistent with the presence of very distant elements in select scenes.

mean_depth and **median_depth:** Indicate a relatively consistent central tendency across images, though both show significant outliers.

std_depth: Highlights wide variability in depth structure, with a heavy concentration of outliers for scenes with high spatial complexity.

depth_range: Displays substantial spread and the most extreme outliers of all metrics, underlining the diversity of depth compositions in the dataset.

3.2.3. Maximum Depth Metric

To better understand the extreme values in the **max_depth** distribution, we examined several of the most significant outliers.

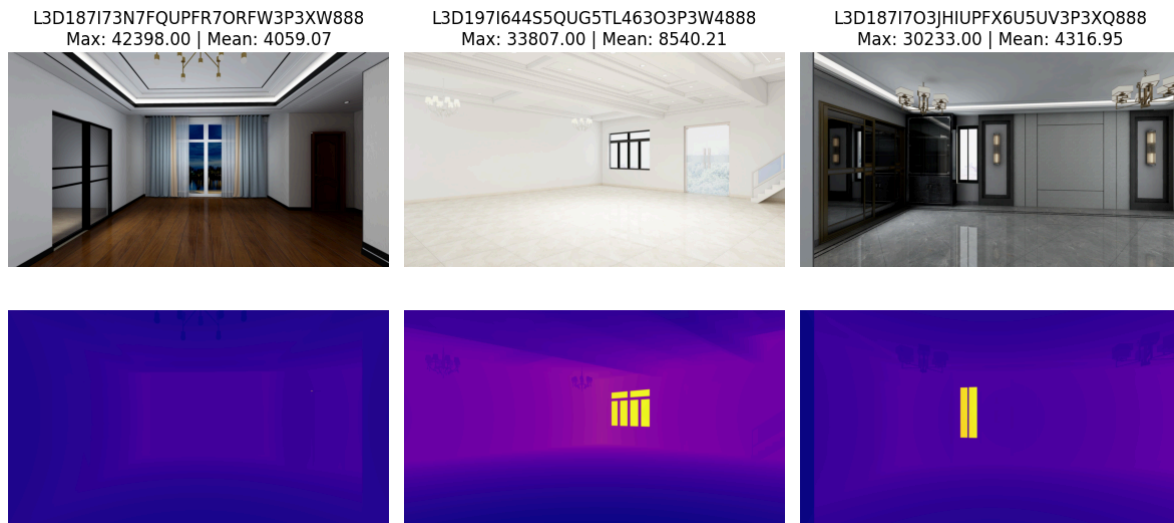


Figure 11: Examples with high maximum depth values

In each case, the RGB images (top row) and their corresponding depth maps (bottom row) reveal that:

- The extremely high depth values are often caused by windows or transparent surfaces, where the depth estimation extends unnaturally far into the background.
- In some instances, only a few isolated pixels contain unusually large values that are inconsistent with the surrounding scene. These are likely due to rendering artifacts or sensor noise.

3.3. Data Cleaning

The data analysis revealed a recurring issue in the dataset: certain depth maps contain extremely high depth values, often well beyond the expected range for indoor scenes. These anomalies are most commonly found near windows or transparent surfaces, where the depth renderer appears to return values corresponding to far-away backgrounds. In some cases, only a few isolated pixels exhibit these abnormal values, likely due to rendering artifacts or sensor inconsistencies.

These extreme values are problematic, especially when training models like Depth Anything assume a maximum depth of 20 meters by default. In our dataset, this corresponds to a numeric threshold of 20,000 units. Values exceeding this limit may cause the model to misinterpret the scene geometry, skew learning, or reduce generalization.

To address this, we propose two main strategies:

1. Threshold-Based Filtering (Simple and Effective)
2. In-Image Depth Cleaning (Advanced, Targeted)

3.3.1. Threshold-Based Filtering

The simplest approach is to exclude any depth maps with a maximum depth greater than 20,000 (20 meters) from the dataset. This prevents outliers from influencing the training process and aligns the dataset with the expected input range of most depth estimation models.

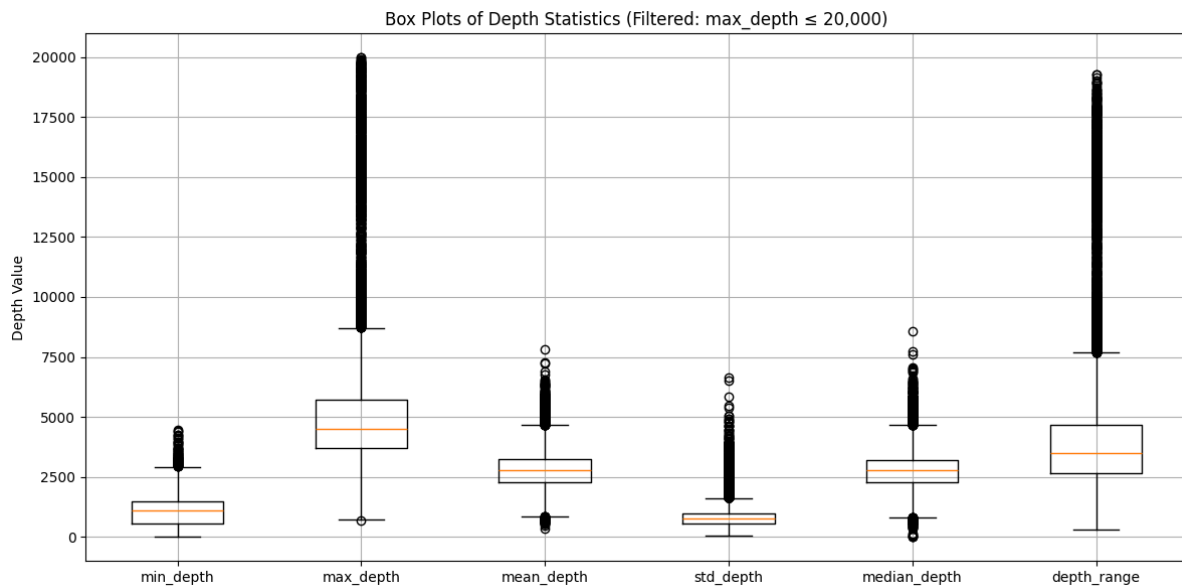


Figure 12: Box plots after data cleaning with threshold filtering.

After applying threshold filtering, the dataset contains significantly fewer outliers than before. We decided to retain the remaining outliers and will perform additional data cleaning if the model demonstrates poor performance.

3.3.2. In-Image Depth Cleaning

Upon initial inspection of the DeepFurniture dataset, we identified a significant data quality issue in scenes containing windows. The depth maps in these regions often exhibited extreme depth values, which is common for transparent or highly reflective surfaces where depth sensors like to fail. These anomalous values severely degraded the performance of our fine-tuned depth estimation model during training.

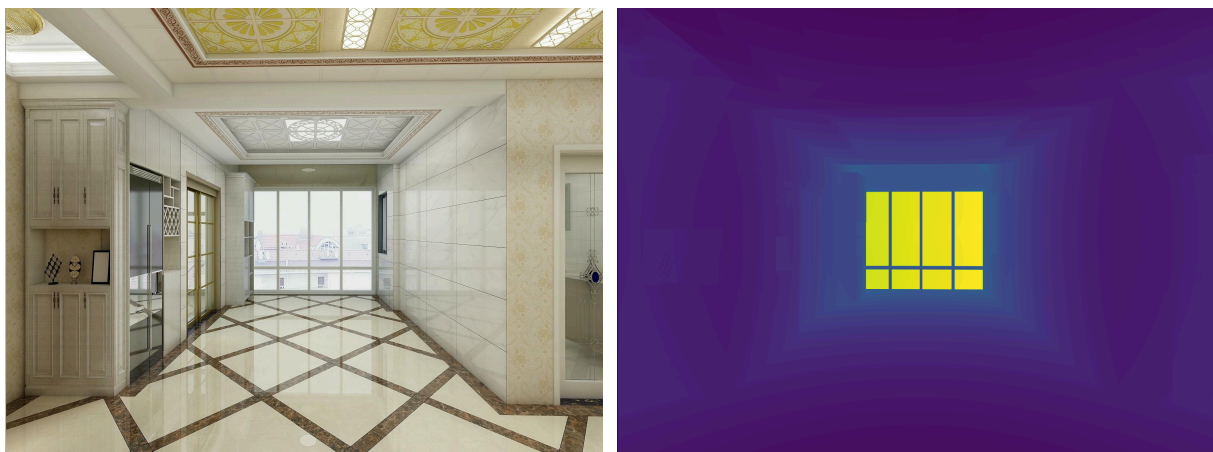


Figure 13: Left: original image; Right: original depth map

To address this issue, we developed a preprocessing pipeline to identify window regions and correct their depth values. Our approach assumes that the depth of a window should be consistent with the depth of the surrounding wall. We explored two distinct methods for this correction task.

3.3.2.1. Method 1: Semantic Segmentation

Our first approach was to explicitly detect windows using a pre-trained semantic segmentation model [10]. We utilized the nvidia/segformer-b5-finetuned-ade-640-640 model [11], a powerful vision transformer fine-tuned on the ADE20K dataset [12]. The process was as follows:

1. **Inference:** For each scene’s RGB image, the Segformer model was used to generate a semantic segmentation map.
2. **Mask Generation:** We created a binary mask by isolating the pixels classified as “window” or “windowpane” (class IDs 8 and 21 in the ADE20K label set).
3. **Boundary Analysis:** The generated window mask was then eroded by a few pixels. The boundary region (the original mask minus the eroded mask) was assumed to represent the wall area immediately surrounding the window.
4. **Depth Correction:** The median depth value of the pixels in this boundary region was calculated. This median wall depth was then used to fill the entire window area in the original depth map, creating a corrected version.



Figure 14: Left: generated mask; Right: mask applied to depth map

While theoretically sound, this method proved to be suboptimal. The pre-trained segmentation model struggled to reliably and accurately identify windows across the diverse scenes of the DeepFurniture dataset, often resulting in incomplete or incorrect masks. This unreliability led us to explore a more direct and robust alternative.

3.3.2.2. Method 2: Direct Depth-Based Thresholding

Given the shortcomings of the segmentation-based method, we implemented a simpler yet more effective approach that operates directly on the depth maps. This method is based on the observation that anomalous window depths are almost always the largest values in a given scene’s depth map.

1. **Thresholding:** For each depth map, we calculated a high-percentile depth value from all valid (non-zero) pixels. Any pixel with a depth greater than this threshold was considered part of a potential window region.
2. **Mask Cleaning:** This initial depth mask was often noisy. To get a cleaner shape, we applied morphological operations. First, a “binary closing” filling small holes and then a “binary opening” removing any small, isolated noise patches.
3. **Depth Correction:** With this cleaned window mask, we applied the same boundary analysis and median-fill technique as in the first method. The boundary pixels of the mask were used to determine the surrounding wall’s depth, which then filled the entire window region in the depth map.

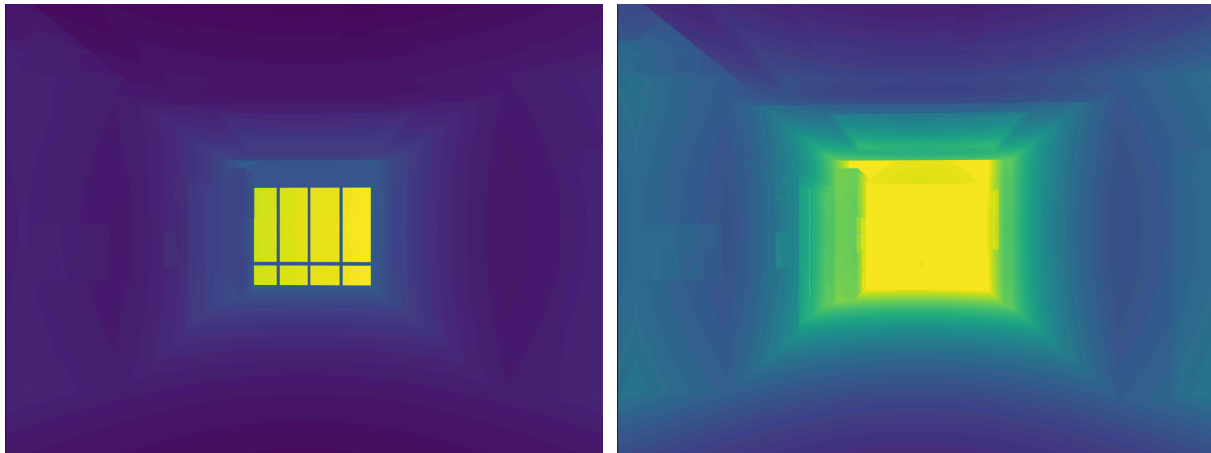


Figure 15: Left: original depth map; Right: corrected depth map

This direct depth correction method proved significantly more reliable and robust for our fine-tuned model. By sidestepping the challenges of semantic understanding and instead leveraging the distinct numerical properties of the depth data itself, we consistently corrected the anomalous window depth values, thereby improving the overall quality of the dataset for fine-tuning the model. All subsequent experiments were conducted using the preprocessed depth maps using this second method.

3.4. Preprocessing

Preprocessing was performed prior to training to reduce computational overhead during model execution and to enable full utilization of GPU resources. In particular, each sample's RGB image was resized and normalized, while the corresponding depth map was also resized, scaled from millimeters to meters, and thresholded to exclude outlier values above 20 meters.

The processed image-depth pairs were stored in an HDF5 format, which offers a balance between disk I/O performance and memory efficiency. This format allows the data to be lazily loaded during training, preventing memory overflow while maintaining fast access speeds. Each subset (train, validation, test) was saved as a separate HDF5 file, with individual samples organized into compressed datasets to further optimize storage and retrieval performance.

3.5. Data Splitting

After data cleaning and preprocessing, the dataset was randomly split into training, validation, and test subsets with a ratio of 80% / 10% / 10%. This stratified split was applied at the sample level, maintaining a balanced distribution across all data partitions.

4. Methods

4.1. SfM/MVS Pipeline

The SfM/MVS pipeline consists of two primary stages: Structure from Motion for camera pose estimation and sparse reconstruction, followed by Multi-View Stereo for dense geometry recovery. This sequential approach transforms multiple overlapping images into detailed 3D models through systematic geometric analysis and optimization. The information on the pipeline is mainly from the HSLU's module "Computer Vision and Artificial Intelligence". [13]

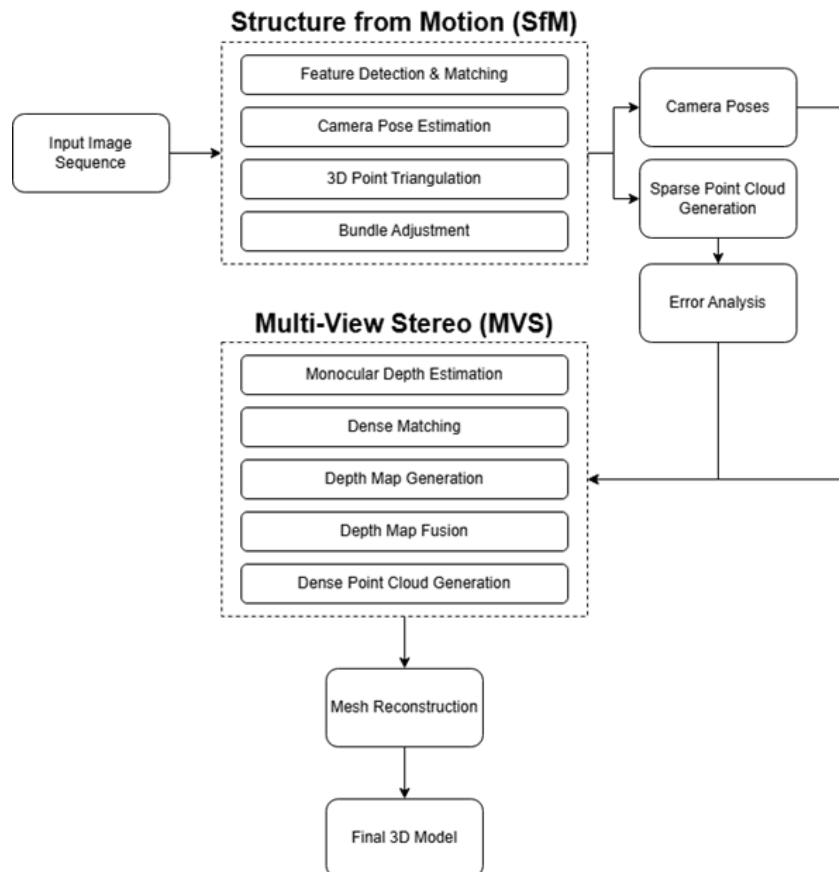


Figure 16: SfM/MVS pipeline

4.1.1. Structure from Motion (SfM)

4.1.1.1. Input Requirements

SfM requires multiple photographs captured from different viewpoints with sufficient overlap to establish feature correspondences. Optimal results demand 60-80% overlap between adjacent images, ensuring robust feature matching across the image sequence. Camera motion should be smooth and deliberate, avoiding sudden movements that compromise tracking accuracy.

4.1.1.2. Feature Detection and Matching

The pipeline begins with identifying distinctive visual features across all input images. Modern implementations employ scale-invariant feature transform (SIFT), speeded-up robust features (SURF) or oriented FAST and rotated BRIEF (ORB) descriptors to ensure robust matching under varying illumination and perspective conditions.

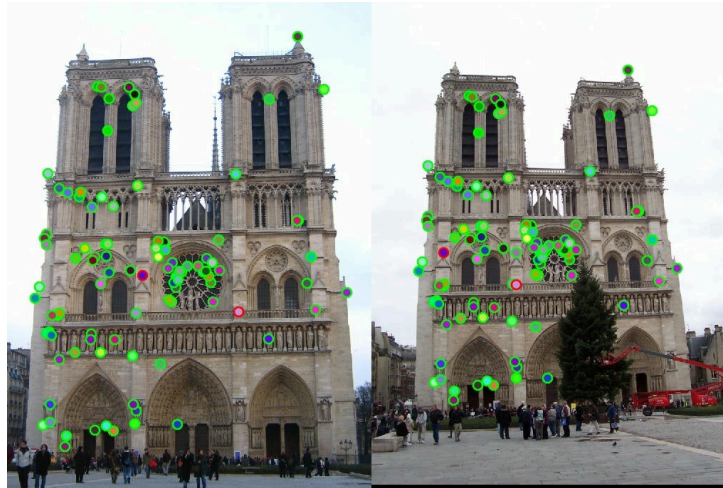


Figure 17: Feature detection

Feature matching establishes correspondences between images by comparing descriptor similarity and enforcing geometric constraints. Robust estimation techniques, including Random Sample Consensus (RANSAC), eliminate outlier matches and ensure correspondence reliability. The resulting feature tracks connect the same physical points across multiple images, forming the foundation for geometric reconstruction.

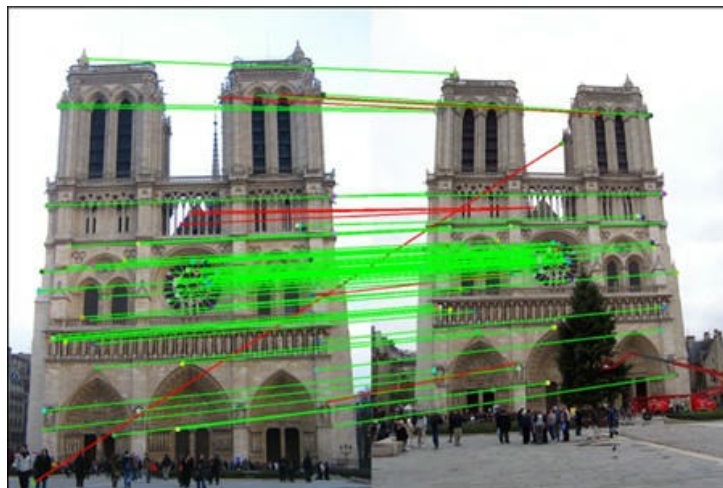


Figure 18: Feature matching

4.1.1.3. Camera Pose Estimation

Camera pose estimation determines the 3D position and orientation of each camera relative to the scene coordinate system. This process employs fundamental matrix and essential matrix estimation for image pairs, followed by pose recovery through singular value decomposition. Initial pose estimation often begins with a robust image pair selection, choosing images with strong feature correspondences and favorable geometric configuration. Subsequent images are incrementally added to the reconstruction through perspective-n-point (PnP) algorithms, progressively expanding the camera network.

4.1.1.4. 3D Point Triangulation

Triangulation converts 2D feature correspondences into 3D world coordinates using known camera poses. For each feature track, rays are projected from corresponding image points through their respective camera centers. The intersection of these rays determines the 3D point location, with geometric uncertainty handled through least-squares optimization.

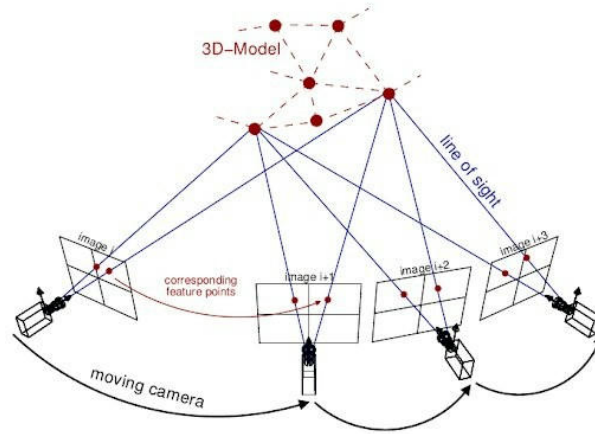


Figure 19: Camera pose estimation and 3D point triangulation

4.1.1.5. Bundle Adjustment

Bundle adjustment represents the cornerstone optimization stage, simultaneously refining camera poses and 3D point positions to minimize reprojection errors across all observations. This non-linear optimization problem employs Levenberg-Marquardt algorithms to achieve convergence while maintaining numerical stability. The optimization objective minimizes the sum of squared reprojection errors: “ $\min \sum_{ij} \|x_{ij} - \pi(P_i, X_j)\|^2$ ”, where x_{ij} represents observed image coordinates, π denotes the camera projection function, P_i represents camera parameters, and X_j denotes 3D point coordinates.

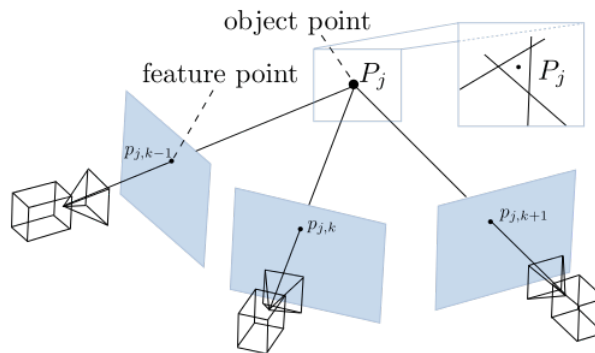


Figure 20: Bundle Adjustment

4.1.1.6. Sparse Point Cloud Generation

The SfM stage produces a sparse point cloud containing only reliably reconstructed feature points. This initial reconstruction captures the scene’s geometric structure and camera configuration, providing the foundation for subsequent dense reconstruction. Quality assessment involves analyzing reprojection errors, track lengths, and geometric consistency across reconstruction.

4.1.2. Multi-View Stereo (MVS)

4.1.2.1. Depth Estimation

Modern MVS pipelines incorporate monocular depth estimation (MDE) to enhance reconstruction quality and completeness. Deep learning models, including MiDaS (hybrid CNN-Transformer) and DepthAnything (Vision Transformer), provide pixel-wise depth predictions for individual images.

Integration of MDE results with geometric reconstruction improves surface coverage and reduces reconstruction artifacts in textureless regions.



Figure 21: Monocular Depth Estimation

4.1.2.2. Dense Matching

Dense matching extends correspondence establishment from sparse features to pixel-level accuracy. This process employs photometric consistency measures, including normalized cross-correlation (NCC) and zero-mean normalized cross-correlation (ZNCC), to identify corresponding pixels across multiple views. Patch-based matching algorithms analyze local image neighborhoods to establish robust correspondences while handling photometric variations and geometric distortions. Advanced approaches employ cost-volume construction and optimization to achieve sub-pixel matching accuracy.

4.1.2.3. Depth Map Generation

Individual depth maps are generated for each input image through multi-view stereo matching. The process involves:

- **Reference Image Selection:** Each image acts as a reference, with neighboring images as sources
- **Cost Volume Construction:** Photometric costs are computed across depth hypotheses
- **Cost Aggregation:** Spatial consistency is enforced through filtering operations
- **Depth Optimization:** Winner-takes-all or global optimization selects optimal depth values

4.1.2.4. Depth Map Fusion

Multiple depth maps must be fused into a consistent 3D representation to handle overlapping coverage and resolve conflicts between different viewpoints. Fusion algorithms consider depth uncertainty, viewing angle effects, and photometric confidence to weight contributions from different maps. Truncated Signed Distance Function (TSDF) fusion represents a popular approach, maintaining a volumetric grid where each voxel stores distance-to-surface measurements. This representation naturally handles multiple observations and produces smooth, consistent surfaces.

4.1.2.5. Dense Point Cloud Generation

Dense matching results are converted to 3D coordinates through triangulation, producing point clouds with significantly higher density than the initial sparse reconstruction. Quality filtering removes points with high geometric uncertainty or poor photometric consistency.

The resulting dense point cloud contains millions of 3D points representing detailed surface geometry, capturing fine-scale features and object boundaries with high fidelity.

4.1.2.6. Mesh Reconstruction

Surface reconstruction transforms dense point clouds into continuous polygon meshes suitable for rendering and visualization. Poisson reconstruction represents the most widely adopted approach, solving for a smooth surface that best fits the oriented point data. The Poisson formulation treats surface reconstruction as solving: “ $\nabla \cdot \nabla \chi = \nabla \cdot \mathbf{V}$ ”, where χ represents the indicator function and $\mathbf{V}$ denotes the vector field derived from oriented points.

4.2. Monocular Depth Estimation

Monocular depth estimation is the process of inferring a dense depth map where each pixel in an image has a depth value. It uses only one RGB image and does not rely on stereo or multi-view input. Because a single image lacks explicit 3D information, the task depends on visual cues and prior knowledge about the structure of the world.

4.2.1. Absolute and Relative Depth

Depth information can be represented in two primary forms: absolute (or metric) and relative. The distinction between these is crucial for understanding the capabilities and limitations of different depth estimation models and datasets. [14]

4.2.1.1. Absolute (Metric) Depth

Absolute depth refers to the physical distance between the camera sensor and an object or point in the scene, measured in real-world units like meters or centimeters. It is typically captured by active sensors that can directly measure depth, such as LiDAR scanners, Time-of-Flight (ToF) cameras, Structured light sensors (like the Microsoft Kinect), and stereo camera setups with a known distance between cameras.

Datasets providing absolute depth, such as DepthAnything V2, serve as essential ground truth for training and evaluating depth estimation models. A model that predicts absolute depth can be directly used for tasks requiring real-world scale, such as robotics, augmented reality, and accurate 3D measurement.

4.2.1.2. Relative Depth

Relative depth, in contrast, does not have a true, meaningful scale. It preserves the depth ordering and structure of the scene, meaning it can correctly determine that one object is closer than another, but the output values are not tied to any physical unit. For example, a relative depth map might indicate that one pixel has a depth value of 0.5 and another has a value of 0.2, but this only implies a relative distance, not a specific measurement in meters.

4.2.2. Loss Functions

Training a depth estimation model on diverse datasets or for the general task of relative depth prediction, presents a significant challenge: the ground truth depth maps often lack a consistent scale and shift. A standard regression loss like L2 or L1 would be skewed by these variations. To overcome this, we employ a loss function invariant to these affine transformations, compelling the model to learn the underlying geometric structure of the scene rather than dataset-specific scaling conventions.

The core of this objective is an affine-invariant Mean Absolute Error (MAE) loss computed in disparity space. First, depth values t are converted to disparity d via the relation $d = \frac{1}{t}$. Then, instead of directly comparing the predicted disparity map $\mathbf{d}^*$ and the ground truth disparity map $\mathbf{d}$, each map is independently normalized to have a zero median (shift-invariance) and a unit mean absolute deviation (scale-invariance).

The loss L is defined as the MAE between these aligned maps:

$$L = \frac{1}{HW} \sum_{i=1}^{HW} |\hat{d}_i^* - \hat{d}_i|$$

where $\hat{d}_i^*$ and $\hat{d}_i$ are the aligned prediction and ground truth values for pixel i . The alignment for any given disparity map $\mathbf{d}$ is performed as follows:

$$\hat{d}_i = \frac{d_i - t(\mathbf{d})}{s(\mathbf{d})}$$

The term $t(\mathbf{d})$ provides shift-invariance by centering the distribution at zero using the median, which is robust to outliers:

$$t(\mathbf{d}) = \text{median}(\mathbf{d})$$

The term $s(\mathbf{d})$ provides scale-invariance by normalizing the map's spread using the mean absolute deviation:

$$s(\mathbf{d}) = \frac{1}{HW} \sum_{i=1}^{HW} |d_i - t(\mathbf{d})|$$

By computing the loss on these normalized maps, the model is penalized only for differences in structure, not for the scale and shift, which can be arbitrary across different scenes and datasets. This base loss can be further augmented with other terms, such as a gradient-matching component, to better enforce the preservation of sharp details and high-frequency textures. [15]

4.2.3. Models

To start with the best possible model as a base to build up on, we had to evaluate them on the DeepFurniture Dataset. For this, we use the two state-of-the-art models, MiDaS and Depth Anything V2, in the biggest publicly available version.

4.2.3.1. MiDaS

MiDaS is a Monocular Depth Estimation model built by Intel. A key characteristic of MiDaS is its training methodology, which involves leveraging multiple diverse datasets (up to 10 in some versions, including a mix of LiDAR, stereo and 3D film data). To handle the inherent differences in scale and shift across these varied datasets, MiDaS employs a scale and shift-invariant loss function during training. This approach enables the models to generalize well to unseen environments and produce high-quality relative depth maps from single RGB images. Several versions of MiDaS exist, offering different trade-offs between accuracy and computational cost, often utilizing transformer-based (e.g., DPT) or convolutional backbones.

4.2.3.2. Depth Anything V2

Depth Anything V2 [16] is a more recent and powerful model for monocular depth estimation, building upon its predecessor, Depth Anything [17]. A core strategy in Depth Anything V2 involves a sophisticated training pipeline emphasizing data quality and scale. Notably, it replaces labeled real images from its V1 training with high-quality synthetic images for initial teacher model training. This teacher model, often with a scaled-up capacity (e.g., using DINOv2-G encoders), is then used to generate pseudo-labels for a vast corpus of unlabeled real-world images (around 62 million). The final student models, available in various sizes (e.g., Small, Base, Large) to cater to different application needs, are trained on these pseudo-labeled real images. This multi-stage approach, combining precise synthetic data with large-scale, diverse real-world imagery via a teacher-student paradigm, allows Depth Anything V2 to produce exceptionally robust depth predictions with fine-grained details across a wide array of scenes. It aims to provide a versatile solution for relative depth estimation, and can also be fine-tuned for metric depth.

4.2.3.3. Comparison

Model	AbsRel ↓	$\delta_1 \uparrow$	$\delta_2 \uparrow$	$\delta_3 \uparrow$
Midas	0.1464	0.8446	0.9601	0.9816
DepthAnythingV2	0.1755	0.8862	0.9719	0.9864

Table 1: Performance measures of pretrained and finetuned DepthAnythingV2 (Beyond2D) models on DeepFurniture dataset. The metrics include AbsRel ↓ (lower is better), and the threshold accuracies δ_1 , δ_2 , and $\delta_3 \uparrow$ (higher is better).

The results present a nuanced comparison between the two models on relative depth. The predictions were scaled to the ground truth to make both models comparable.

Notably, MiDaS achieved a lower (better) Absolute Relative Error, suggesting its average per-pixel error is smaller across the dataset. However, Depth Anything V2 demonstrated superior performance across all other key metrics.

Depth Anything V2 achieved a significantly lower RMSE, indicating that it produces fewer large errors than MiDaS. This is a critical advantage, as large errors can have a more detrimental impact on 3D reconstruction tasks. Furthermore, its superiority is confirmed by the delta accuracy metrics (δ_1 , δ_2 , δ_3), where it consistently outperformed MiDaS. The most significant gap is observed in the δ_1 accuracy, with Depth Anything V2 correctly predicting the depth of 88.6% of pixels within the strictest threshold, compared to 84.5% for MiDaS.

In conclusion, while MiDaS shows a strong performance in relative error, Depth Anything V2 proves to be the more robust and reliable base model for the DeepFurniture dataset. Its ability to avoid major prediction errors (lower RMSE) and its higher overall pixel accuracy (higher δ values) make it a stronger foundation model for our fine-tuning task. This suggests that Depth Anything V2’s internal representations are better suited for complex indoor scenes and furniture geometries present in this dataset.

5. Results

In this project, we introduce Beyond2D, a depth refinement framework designed to enhance monocular absolute depth estimation from pretrained models. As 3D reconstruction requires accurate depth estimation we built on the capabilities of Metric DepthAnythingV2 (absolute depth) and demonstrate consistent improvements in both quantitative metrics and qualitative outputs. Our method is evaluated on the DeepFurniture dataset, where Beyond2D-fine-tuned models outperform their pretrained counterparts in key performance metrics, including AbsRel and threshold accuracies δ_1 , δ_2 , and δ_3 . Additionally, we present visualizations that highlight the refinement in depth predictions, confirming the effectiveness of Beyond2D in producing more accurate and structured depth maps.

5.1. Performance

Model	Encoder	AbsRel ↓	δ_1 ↑	δ_2 ↑	δ_3 ↑
Metric DepthAnythingV2	ViT-S	0.9090	0.1524	0.5477	0.8142
Beyond2D - Metric DepthAnythingV2	ViT-S	0.2230	0.8375	0.9561	0.9811
Metric DepthAnythingV2	ViT-L	0.8728	0.1738	0.5994	0.8407
Beyond2D - Metric DepthAnythingV2	ViT-L	0.1309	0.9080	0.9687	0.9848

Table 2: Performance measures of pretrained and finetuned DepthAnythingV2 (Beyond2D) models on DeepFurniture dataset. The metrics include AbsRel ↓ (lower is better), and the threshold accuracies δ_1 , δ_2 , and δ_3 ↑ (higher is better).

For the ViT-S backbone, the pretrained Metric DepthAnythingV2 model achieves an AbsRel of 0.9090, with δ_1 , δ_2 and δ_3 scores of 0.1524, 0.5477 and 0.8142. After fine-tuning with Beyond2D, the model achieves improved performance with an AbsRel of 0.2230 and δ_1 , δ_2 and δ_3 values of 0.8375, 0.9561, and 0.9811.

For the ViT-L backbone, the pretrained Metric DepthAnythingV2 model achieves an AbsRel of 0.8728, with δ_1 , δ_2 and δ_3 values of 0.1738, 0.5994 and 0.8407. The Beyond2D-finetuned variant shows further improvement, reaching an AbsRel of 0.1309 and threshold accuracies of 0.9080, 0.9687 and 0.9848, respectively.

5.2. Visualizations

In the following visualizations, we compare the ground truth depth with the pretrained and finetuned prediction output.

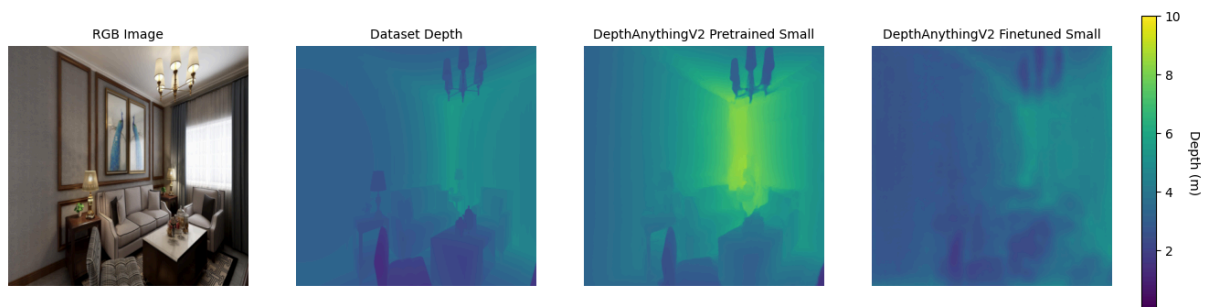


Figure 22: DepthAnythingV2-Small depth map comparison

The figure above compares depth predictions from pretrained and fine-tuned DepthAnythingV2 models using the ViT-S backbone on a sample from the DeepFurniture dataset. The leftmost panel shows the input RGB image, followed by the ground truth depth map from the dataset. The third panel presents the output of the pretrained DepthAnythingV2 (ViT-S), which captures structural details well but tends to overestimate depth, producing values beyond the actual scene range. The final panel displays the output from the Beyond2D-fine-tuned model. While this prediction lies closer to the correct depth range, it suffers from a noticeable loss of spatial detail and appears overly smooth and blurred.

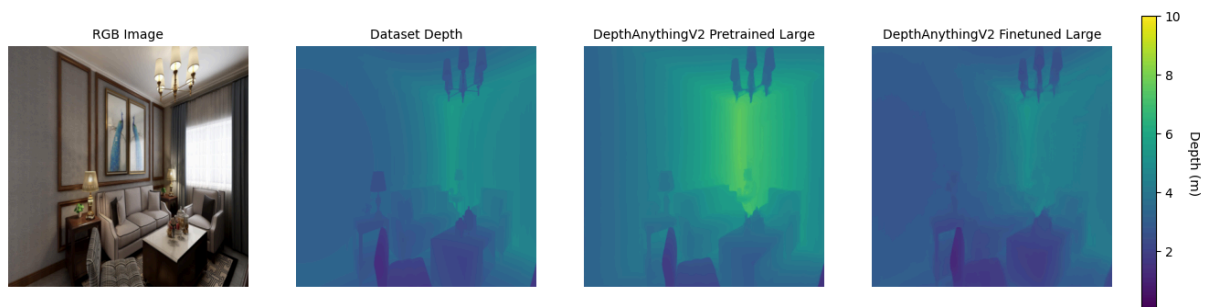


Figure 23: DepthAnythingV2-Large depth map comparison

In this example, using the ViT-L encoder, the pretrained model again overestimates depth values but retains well-defined object boundaries. The fine-tuned model produces depth predictions that closely match the ground truth in range, while also recovering more scene details compared to the ViT-S variant. However, some fine structures remain imperfectly resolved.

6. Model Integration

To see how well our fine-tuned DepthAnythingV2 model performed in practice, we decided to integrate it into an existing 3D reconstruction pipeline. We chose COLMAP as our testing ground because of its modular design, which gave us the flexibility to swap out specific components. This meant we could replace the traditional geometric depth estimation methods with our AI-powered approach while keeping everything else stable and reliable.

6.1. How COLMAP Works

COLMAP is a comprehensive toolkit for turning a set of images into detailed 3D models. It works through several distinct steps, which can also be seen in the SfM/MVS chapter: first extracting features from images, then matching those features between different photos, creating a sparse 3D structure and finally building a dense, detailed reconstruction.

The process starts by finding and matching SIFT features across all the input images. Those can be imagined as distinctive visual landmarks that the system can recognize from different angles. Once these matches are established, COLMAP uses incremental SfM techniques to figure out where the camera was positioned for each photo and creates a basic 3D skeleton of the scene. The final step uses patch-match stereo algorithms to fill in all the details, generating comprehensive depth maps for every viewpoint.

What comes out of this process are two main products: a sparse point cloud with thousands of reliable 3D points that capture the basic structure of the scene, and a dense point cloud that can contain millions of points representing every surface detail possible.

While COLMAP is a powerful tool for 3D reconstruction, it can be prone to instability under certain conditions. The system can become quite unstable when handling large image sets, often struggling to maintain consistent feature matching across extensive collections. Image spacing is another challenge. Photos must be taken from positions close enough to ensure sufficient overlap for reliable feature detection. When viewpoints are too far apart, overlap decreases and leads to poor matching performance. There are also some issues when using the sequential matcher: a single poorly matched image can halt the entire reconstruction process. To bypass this problem, the exhaustive matcher can be used, which doesn't check for the perfect order of the images. In general, the COLMAP pipeline could still be improved by fine-tuning every parameter perfectly for this task.

6.2. Integration Approach

We integrated our DepthAnythingV2 model right at the dense reconstruction stage, essentially replacing COLMAP's patch-match stereo depth estimation with our neural network predictions. This strategy allowed us to get the best of both worlds: we could still rely on COLMAP's proven geometric methods for figuring out camera positions, while leveraging modern deep learning for the actual depth estimation.

We first let COLMAP run its standard patch-match stereo process to set up the dense reconstruction framework, then we carefully replace those geometric depth maps with our DepthAnythingV2 predictions. This two-step approach ensures that everything plays nicely with COLMAP's existing fusion and meshing algorithms, while giving us the enhanced depth estimation we were after. The key was making sure our neural depth maps used the same spatial resolution and coordinate system as the original estimates, so the integration would be seamless.

6.3. Results Basic DepthAnythingV2

Our experiments with the pretrained DepthAnythingV2 model (before any custom training) already showed some improvements over the traditional COLMAP reconstruction. Our chosen monocular depth estimation model already detected more feature points, which the traditional COLMAP pipeline couldn't detect. The model slightly improved on blank walls, textureless surfaces and also on smaller details, which are hard to match without a good depth estimation.

In the scene below, which was taken in a real living room, some minor improvements can be observed. For example, the gap in the wall got a bit smaller and the whole wall seems more complete. On the right-hand side, the grey curtain also seems fuller. The metal stand structure, which can be seen in the front on the right side, appears more stable and complete, with less fragmentation in the point cloud reconstruction. The same goes for the two plants on and under the metal stand. Overall, the number of points was doubled (from 231'952 to 491'578), which indicates that this approach is already working and successfully generating denser reconstructions.



Figure 24: Left: Real picture of the scene; Middle: Traditional COLMAP approach; Right: COLMAP with pretrained DepthAnythingV2

On the second scene that we tested, we could already see some bigger improvements. The whole scene seems much fuller with the integrated DepthAnythingV2 depth estimation. This can also be observed in the number of generated points: The traditional COLMAP pipeline generated 255'566 points, whereas our adjusted MVS generated 524'738 points - so again almost twice as much. This is especially visible in the left corner of the room and also on the table. The noise also got a bit worse, but this is due to the fact that the SfM approach has some problems with reflections in the scene.

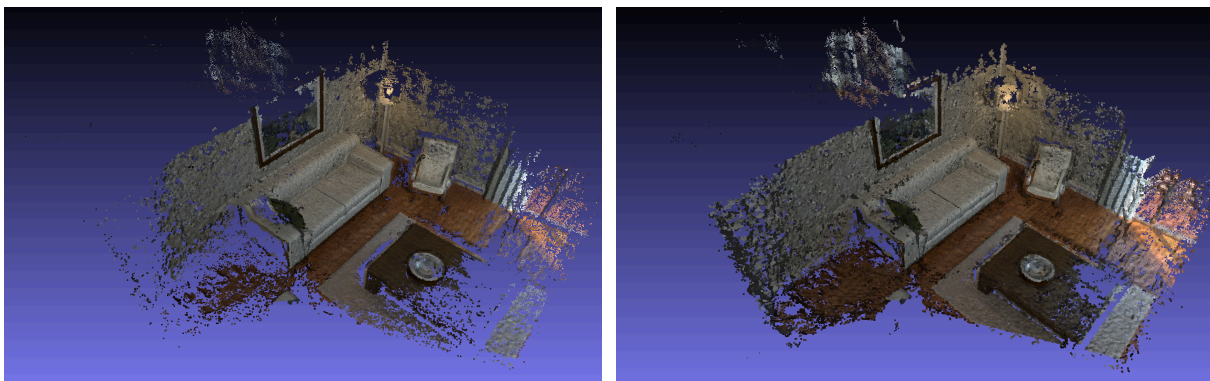


Figure 25: Left: Traditional COLMAP approach; Right: COLMAP with pretrained DepthAnythingV2

6.4. Results Trained DepthAnythingV2

Our fine-tuned DepthAnythingV2 model, which we trained specifically on the DeepFurniture dataset, showed minor improvements over the pretrained version. It performed a bit better on textureless surfaces and small details. The number of feature points also increased again, which indicates some improvements.

In the scene from the real living room, the gap in the wall decreased once more, which shows the above-mentioned performance boost on the textureless areas. Besides this, the two scenes appear nearly identical, which is also reflected in the feature point counts: The pipeline using the pretrained model produced 491'578 points, while the fine-tuned version generated only a marginally higher count of 491'608 points.

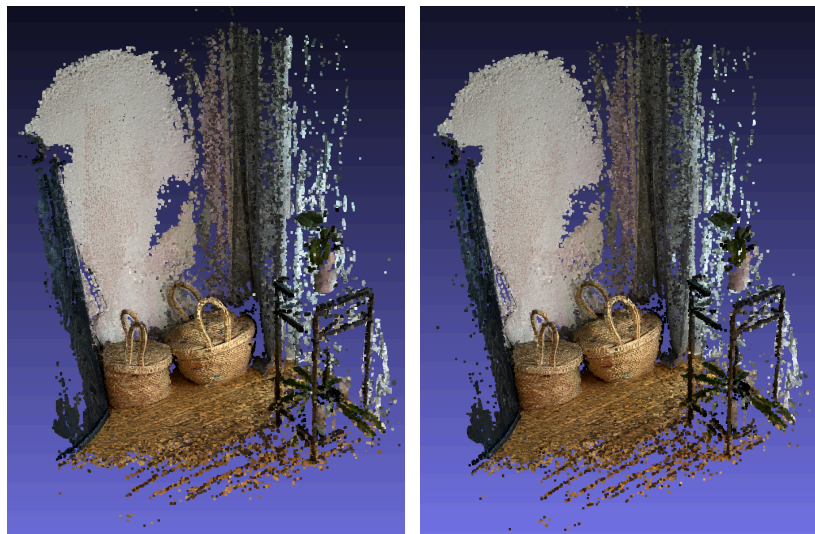


Figure 26: Left: Traditional COLMAP approach; Right: COLMAP with pretrained DepthAnythingV2

For the second scene with the synthetic living room, the improvements are more noticeable. The left corner of the room looks denser and more complete. The balcony on the right side also became more dense with the fine-tuned model. The feature points increased from 524'738 to 609'938, which shows better scene coverage, though this also resulted in more noise being captured along with the additional detail.

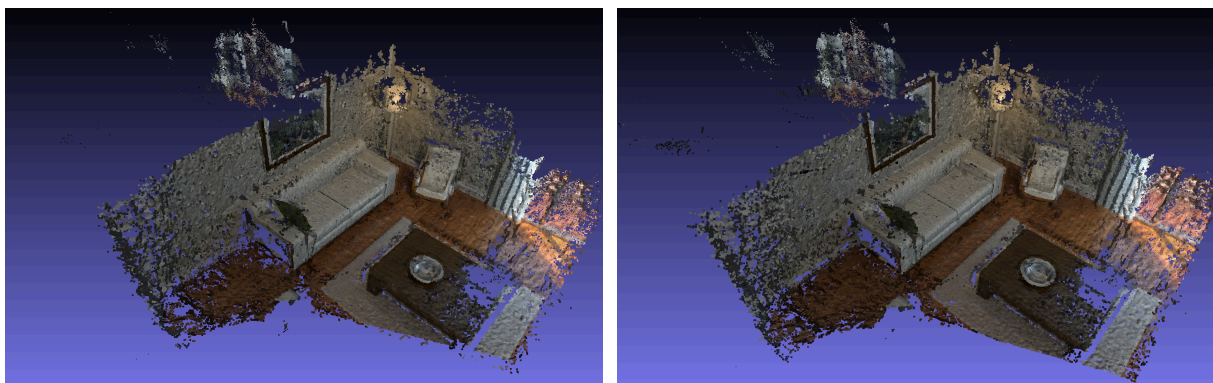


Figure 27: Left: Traditional COLMAP approach; Right: COLMAP with pretrained DepthAnythingV2

6.5. Cleaning Up the Results

Even with all these improvements from the inserted depth estimation, the reconstructed point clouds still needed additional post-processing to reach production quality standards. We developed a Python-based filtering algorithm to tackle the most common issues: removing geometric outliers and reducing noise artifacts.

Our statistical outlier removal process eliminated points with extreme depth values that were clearly wrong, while radius-based filtering got rid of isolated points that didn't have enough neighboring points to be reliable.

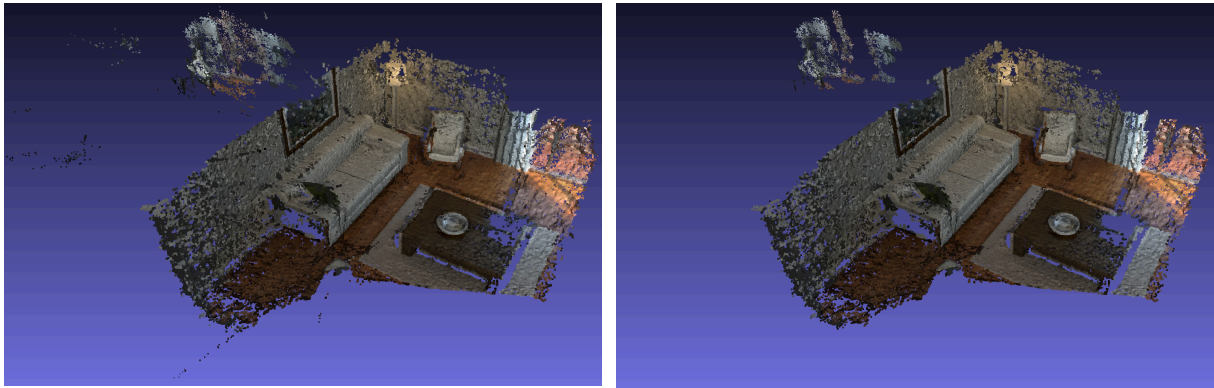


Figure 28: Left: Uncleaned scene; Right: Cleaned with Python-based algorithm

These automated processing steps made a significant difference in visual quality, though we found that some manual point removal was still necessary to achieve perfectly clean models suitable for professional visualization work.

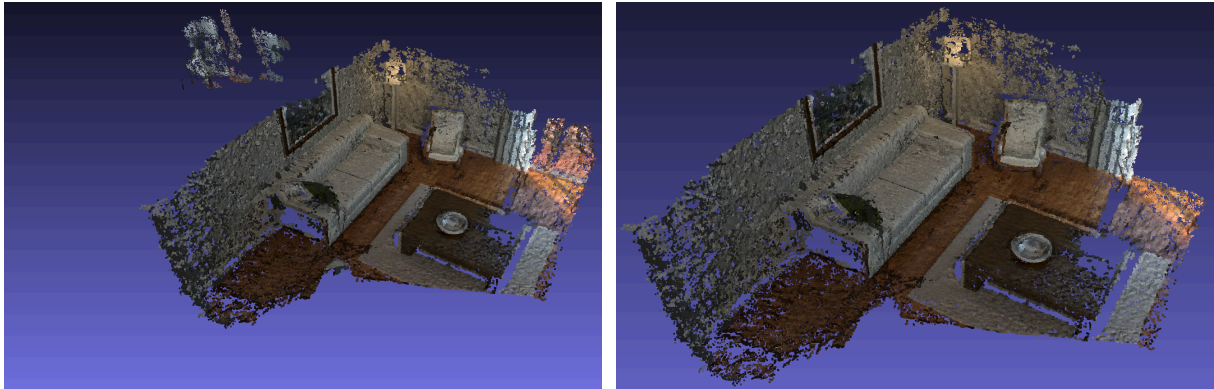


Figure 29: Left: Cleaned with Python-based algorithm; Right: Manual point removal

6.6. Creating the Final Meshes

The last step in our reconstruction pipeline involved converting those dense point clouds into polygon meshes using Poisson surface reconstruction. The resulting meshes did a great job of capturing the room geometry and furniture arrangements.



Figure 30: Created mesh from cleaned-up dense point cloud

However, even our enhanced process wasn't perfect. The meshes typically contained various geometric artifacts requiring substantial manual cleanup to achieve professional-quality results. Common issues included surface discontinuities at object boundaries, geometric noise in areas with low texture and incomplete surface coverage in spots where objects blocked the view.

While our enhanced depth estimation reduced these problems compared to traditional methods, getting to perfectly cleaned architectural models suitable for virtual tours still required significant post-processing work.

6.7. Putting Together Complete Apartments

Reconstructing entire apartments from individual room captures could be effectively approached through a multi-scene integration strategy. In this workflow, each room or scene would be reconstructed separately, using a detailed depth estimation pipeline. Later, they would be combined within a 3D modeling environment to form a comprehensive apartment model.

This method offers a practical alternative to attempting a full-apartment reconstruction in a single pass, which can be limited by occlusions, large-scale alignment errors or processing constraints (e.g. too many images for COLMAP). The integration could rely on aligning rooms using architectural features such as shared walls, doorways or floor plans, followed by refinement steps to ensure geometric consistency at the seams. Such a pipeline would allow scalable, modular reconstruction while preserving high geometric fidelity at the room level.

7. Machine Learning Operations

7.1. Version Control

The entire project was managed under a version control system to ensure full traceability. All code for data preprocessing, model training, and evaluation was versioned using Git and stored in a public GitHub repository.

The repository can be accessed at <https://github.com/ynkf/HSLU.DSPRO2.Beyond2D>.

7.2. Cloud Computing

Our experiments, particularly the fine-tuning of large depth estimation models, required significant computational resources. We utilized the computing infrastructure provided by the HSLU, which was accessed via the centrally managed GPUHub, also known as JupyterHub platform. This environment provided on-demand access to powerful hardware for rapid iteration and model training.

7.3. Experiment Tracking

We use Weights & Biases (W&B) for experiment tracking. For each training and validation run, we logged the following information to the W&B platform:

- **Hyperparameters:** learning_rate, weight_decay, warmup_epochs, scheduler_rate, mixed_precision
- **Performance Metrics:** loss, absolute relative error, delta1
- **Model Artifact:** The trained model weights were saved as W&B artifacts, directly linked to its corresponding experiment.



Figure 31: Metrics from top to bottom: loss, AbsRel, δ_1

8. Conclusions

In this section, we share our perspective on the project and reflect on the results we achieved.

8.1. Interpretation of Results

The results of our experiments show that Beyond2D can significantly improve monocular absolute depth estimation when used to refine pretrained models such as Metric DepthAnythingV2. We observed notable improvements in key metrics, especially in Absolute Relative Error (AbsRel), which decreased substantially for both ViT-S and ViT-L backbones. At the same time, the threshold accuracies (δ_1 , δ_2 , δ_3) also improved consistently, suggesting not only more accurate depth values but also greater reliability within practical error ranges.

Upon closer inspection, we found that the pretrained DepthAnythingV2 models already perform well in capturing object shapes and structural features but tend to overestimate depth values. This likely explains why AbsRel remains high despite relatively good δ_3 scores. It appears that while these models can recognize depth relationships across a scene, they struggle to predict them on the correct absolute scale. By applying Beyond2D, this issue is significantly reduced, resulting in more accurate and well-calibrated depth predictions.

Among the two tested backbones, the ViT-L model shows the most benefit from Beyond2D fine-tuning. This may be due to its greater representational capacity, allowing it to better leverage the refinement process. The qualitative results from the DeepFurniture dataset support this: ViT-L combined with Beyond2D produces depth maps that are closer to the ground truth in scale and preserve more fine structure, although some minor details are still not fully recovered.

That said, we also observed that the fine-tuned models tend to produce smoother and sometimes overly simplified depth maps. This smoothing effect is more noticeable with the ViT-S variant but is also present with ViT-L. This suggests a trade-off: while Beyond2D clearly improves depth accuracy overall, it may also reduce spatial detail in the predictions. It is possible that longer training or additional techniques (e.g. alternative loss functions) could help retain or recover finer geometric features in future work.

8.2. Interpretation of Model Integration

Bringing our fine-tuned DepthAnythingV2 model into the COLMAP pipeline turned out to be a really promising step forward. By switching out COLMAP's traditional patch-match stereo method with our neural depth predictions, we were able to combine the reliability of geometric SfM with the flexibility and detail of modern AI depth estimation.

Our fine-tuned model, optimized for indoor scenes via the DeepFurniture dataset, further refined these results, offering denser reconstructions with higher point counts and slightly improved scene coverage. While this introduced more noise in some cases, our post-processing pipeline, including both automated filtering and manual refinement, helped mitigate these issues and produce cleaner outputs.

Finally, instead of trying to reconstruct whole apartments in one go, we found that capturing and processing individual rooms separately, then combining them later, was a much more manageable and scalable approach. This modular strategy made it easier to control quality and fix alignment issues, especially with COLMAP itself, while still producing highly detailed, consistent reconstructions.

Overall, integrating DepthAnythingV2 into COLMAP gave us a solid upgrade in terms of depth quality and scene completeness. There's still a lot of room for improvement in this task for making high-fidelity 3D reconstruction faster, easier and more robust.

8.3. Future Research Directions

Looking ahead, there are several promising directions that could improve our project even further:

In our research, we came across several papers mentioning that AI models can also be integrated into the Structure from Motion pipeline, particularly in the feature extraction stage. Instead of relying solely on traditional descriptors like SIFT, a learned feature extractor could help to detect and match features on challenging surfaces like blank walls, repetitive textures or low-contrast areas. This step could improve the quality of the sparse reconstruction and even more importantly, of camera poses, making the initial steps of the pipeline more robust and accurate.

Another step that we stumbled upon during the project is that COLMAP offers a lot of flexible functions, which all have fine-tunable parameters. Exploring how they interact with our integrated AI depth model and fine-tuning them for indoor scenes could further boost point density, reduce noise and lead to an overall cleaner outcome.

8.4. Summary

Our project “Beyond2D” successfully addressed the challenge of creating 3D models of apartments from 2D images by enhancing a traditional Structure from Motion (SfM) and Multi-View Stereo (MVS) pipeline with a custom-trained monocular depth estimation model.

Our fine-tuned model demonstrated significant quantitative improvements over their pretrained counterparts, substantially reducing the absolute relative error and increasing threshold accuracies on the DeepFurniture test set. The enhanced model was then integrated into the COLMAP reconstruction pipeline, replacing its traditional depth estimation module. This hybrid approach led to visibly denser and more complete point cloud reconstructions of real-world and synthetic scenes, particularly on textureless surfaces and fine details.

While the integration proved successful and represents a promising direction, the final models still required manual post-processing to achieve production quality. The project concludes that combining classical geometric methods with specialized deep learning models is a powerful and scalable strategy for 3D reconstruction, paving the way for more accurate and accessible virtual property tours.

9. Glossary

Term	Description
BRIEF	Binary Robust Independent Elementary Features
CAD	Computer-Aided Design
FAST	Features from Accelerated Segment Test
GANs	Generative Adversarial Networks
LiDar	Light Detection and Ranging
MDE	Monocular Depth Estimation
MVS	Multi-View Stereo
NCC	Normalized cross-correlation
ORB	Oriented FAST and rotated BRIEF
PnP	Perspective-n-Point
RANSAC	Random Sample Consensus
SfM	Structure from Motion
SIFT	Scale-Invariant Feature Transform
SURF	speeded-up robust features
ToF	Time-of-Flight
TSDF	Truncated Signed Distance Function
ViT-L	Vision Transformer - Large
ViT-S	Vision Transformer - Small
W&B	Weights & Biases
ZNCC	Zero-mean normalized cross-correlation

10. References

- [1] H. Ham, J. Wesley, and H. Hendra, "Computer Vision Based 3D Reconstruction : A Review," *International Journal of Electrical and Computer Engineering (IJECE)*, vol. 9, p. 2394, 2019, doi: 10.11591/ijece.v9i4.pp2394-2402.
- [2] H. Xie, H. Yao, X. Sun, S. Zhou, and S. Zhang, "Pix2Vox: Context-Aware 3D Reconstruction From Single and Multi-View Images," 2019, pp. 2690–2698. doi: 10.1109/ICCV.2019.00278.
- [3] P. Nyimbili, H. Demirel, D. Seker, and T. Erden, "Structure from Motion (SfM) - Approaches and Applications," 2016, p. .
- [4] Y. Yao, Z. Luo, S. Li, T. Fang, and L. Quan, "MVSNet: Depth Inference for Unstructured Multi-view Stereo." [Online]. Available: <https://arxiv.org/abs/1804.02505>
- [5] S. P. Tata and S. Mishra, "3D GANs and Latent Space: A comprehensive survey." [Online]. Available: <https://arxiv.org/abs/2304.03932>
- [6] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis." [Online]. Available: <https://arxiv.org/abs/2003.08934>
- [7] B. Liu, J. Zhang, X. Zhang, W. Zhang, C. Yu, and Y. Zhou, "Furnishing Your Room by What You See: An End-to-End Furniture Set Retrieval Framework with Rich Annotated Benchmark Dataset," *arXiv preprint arXiv:1911.09299*, 2019.
- [8] J. V. Piqueres, "Living Room Dataset," [Online]. Available: https://www.doc.ic.ac.uk/~ahanda/VaFRIC/living_room.html
- [9] P. K. Nathan Silberman Derek Hoiem and R. Fergus, "Indoor Segmentation and Support Inference from RGBD Images," in *ECCV*, 2012.
- [10] G. Csúrká, R. Volpi, and B. Chidlovskii, "Semantic Image Segmentation: Two Decades of Research." [Online]. Available: <https://arxiv.org/abs/2302.06378>
- [11] E. Xie, W. Wang, Z. Yu, A. Anandkumar, J. M. Alvarez, and P. Luo, "SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers," in *Neural Information Processing Systems (NeurIPS)*, 2021.
- [12] B. Zhou, H. Zhao, X. Puig, S. Fidler, A. Barriuso, and A. Torralba, "Scene Parsing through ADE20K Dataset," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 633–641.
- [13] A. Smolic, "HSLU CVAI FS25, 3D Reconstruction." 2025.
- [14] D. Suhoi, "Metric and Relative Monocular Depth Estimation: An Overview. Fine-Tuning Depth Anything V2," 2024, [Online]. Available: <https://huggingface.co/blog/Isayoften/monocular-depth-estimation-guide#scale-and-shift-invariant-loss-%F0%9F%98%8E>
- [15] R. Ranftl, K. Lasinger, D. Hafner, K. Schindler, and V. Koltun, "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-shot Cross-dataset Transfer," in *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 2022, pp. 1623–1637.
- [16] L. Yang *et al.*, "Depth Anything V2," *arXiv:2406.09414*, 2024.
- [17] L. Yang, B. Kang, Z. Huang, X. Xu, J. Feng, and H. Zhao, "Depth Anything: Unleashing the Power of Large-Scale Unlabeled Data," in *CVPR*, 2024.
- [18] B. Zhou, H. Zhao, X. Puig, S. Fidler, A. Barriuso, and A. Torralba, "Semantic understanding of scenes through the ADE20K dataset," *International Journal of Computer Vision (IJCV)*, vol. 127, pp. 302–321, 2019.